Algorithm Design Paradigms

Handout on Divide n Conquer

22 February 2026

Counting Inversions

Given an (1-indexed) array A of n distinct integers, Give an algorithm to count the number of inversions in $O(n \log n)$

An inversion is a pair of i, j such that

- $1 \leq i < j \leq n$
- $A[i] > A[j]$

Hint: Consider a function `def inversions(A: list[int]) -> tuple[int, list[int]]` where we are returning both the number of inversions and sorted list `A`.

Strassen's Algorithm

Given two $n \times n$ matrices A and B, compute their product AB in time $O\left(n^{\log_2 7}\right)$

This is quite important,simple and worth spending some time on. The following extensions are neither.

♠ Assuming $m \geq n$, can you find an $O(m^2 n^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times m$ matrix using the above a subroutine. (Hint: Multiply $\left\lceil \frac{m}{n} \right\rceil^2$ pairs of order-n matrices)

♠ Assuming $m \geq n \geq t$, Can you find an $O(mnt^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times t$ matrix using the above a subroutine. (Hint: Multiply pairs of $t \times t$ matrices)

Merging Arrays

Let $A_1, A_2, ..., A_k$ be k arrays, each of which has been sorted.

These arrays are mutually disjoint, namely, no integer can appear in more than one array.

Design an algorithm to merge the k arrays into one sorted array in $O(n \log k)$ time, where n is the total length of the k arrays.

Note: these arrays may have different lengths.

Max Weight Subarray

Let A be an array of n integers (A is not necessarily sorted). Each integer in A may be positive or negative.

Given i, j satisfying $1 \leq i \leq j \leq n$,

Define sub-array $A[i:j]$ as the sequence $(A[i], A[i+1], ..., A[j])$, and the weight of $A[i:j]$ as $A[i] + A[i+1] + ... + A[j]$.

For example, consider $A = (13, -3, -25, 20, -3, -16, -23, 18)$; $A[1:4]$ has weight 5, while $A[2:4]$ has weight -8.

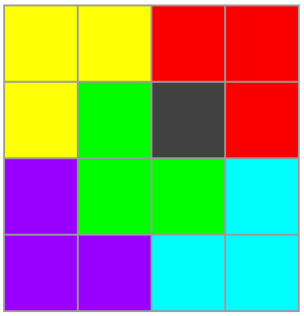
1. Give an algorithm to find a sub-array of with the largest weight, among all sub-arrays $A[i:j]$ with $j = n$. Your algorithm must finish in $O(n)$ time.
2. Give an algorithm to find a sub-array with the largest weight in $O(n \log n)$ time (among all the possible sub-arrays).

L Tiles

Given a grid of $2^N \times 2^N$, all cells are initially empty except one of the cells. Let the blocked cell be B.

Write an algorithm to figure out a way to fill the grid with L shapes (without overlaping other tiles or B) or report "NO" if that is not possible. Your algorithm should run in $O(2^N)$.

The $N = 2, B = (3, 2)$ case is given below.



Hint : We never have to return "NO".

Big Mod

Given p, e, b, compute $p^e \bmod b$ in $O(\log e)$ time.